



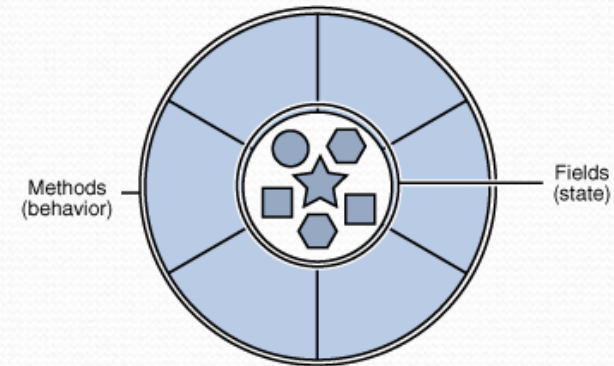
Building Java Programs

Strings

Objects

- **object:** An entity that contains data and behavior.

- *data:* variables inside the object
- *behavior:* methods inside the object
 - You interact with the methods; the data is hidden in the object.
 - A **class** is a *type* of objects.



- Constructing (creating) an object:

Type **objectName** = new **Type** (**parameters**) ;

- Calling an object's method:

objectName.methodName (**parameters**) ;

Strings

- A `String` is an object that contains a sequence of characters.
 - They can be instantiated (created) without using the **new** keyword.
 - They are `immutable`
 - Once instantiated, they are final and cannot be changed.

```
String name = "text";
```

```
String name = expression (with String value);
```

- Examples:

```
String names = "Alice and Bob";
```

```
int x = 3;
```

```
int y = 5;
```

```
String point = "(" + x + ", " + y + ")";
```

Indexes

- Characters of a string are numbered with 0-based *indexes*:

```
String city = "Caloocan";
```

index	0	1	2	3	4	5	6	7
character	C	a	l	o	o	c	a	n

- First character's index : 0
- Last character's index : 1 less than the string's length
- The individual characters are values of type `char` (seen later)

Instantiating a String

- Strings are object reference types:
- They can be instantiated in two ways:
 - The new operator
 - String literals
 - There is no difference between the Strings below.
 - Both methods of instantiation create identical objects.

```
String s1 = new String("abc");    // new operator
String s2 = "abc";                // string literals
```

String References

- When you create a reference to an Object or String, the object does not necessarily exist yet.
- In the code below, since `name` is not initialized, the program will not compile.
- The variable `name` is a `null` pointer.

```
String name;  
System.out.println("My name is " + name) ;
```

`name = null`

String methods

Method name	Description
<code>indexOf(str)</code>	index where the start of the given string appears in this string (-1 if not found)
<code>length()</code>	number of characters in this string
<code>substring(index1, index2)</code> or <code>substring(index1)</code>	the characters in this string from <i>index1</i> (inclusive) to <i>index2</i> (<u>exclusive</u>); if <i>index2</i> is omitted, grabs till end of string
<code>toLowerCase()</code>	a new string with all lowercase letters
<code>toUpperCase()</code>	a new string with all uppercase letters

- These methods are called using the dot notation:

```
String name = "James Gosling";  
System.out.println(name.length());    // 13
```


String method examples

```
public class StringMethods {  
    public static void main(String[] args) {  
        String s1 = "James Gosling";  
        String s2 = "Bill Gates";  
  
        System.out.println(s1.length());           // 13  
        System.out.println(s1.indexOf("e"));       // 3  
        System.out.println(s1.substring(6, 13));   // "Gosling"  
        System.out.println(s1.substring(6));       // "Gosling"  
  
        String s3 = s2.substring(0, 7);  
        System.out.println(s3.toLowerCase());      // "bill ga"  
        System.out.println(s3.toUpperCase());      // "BILL GA"  
    }  
}
```


Modifying strings

- Methods like `substring` and `toLowerCase` build and return a new string, rather than modifying the current string.

```
String s = "Java Programming";  
s.toUpperCase();  
System.out.println(s);    // Java Programming
```

- To modify a variable's value, you must reassign it:

```
String s = "Java Programming";  
s = s.toUpperCase();  
System.out.println(s);    // JAVA PROGRAMMING
```

Strings as user input

- Scanner's next method reads a word of input as a String.

```
Scanner console = new Scanner(System.in);
System.out.print("What is your name? ");
String name = console.next();
name = name.toUpperCase();
System.out.println(name + " has " + name.length() +
    " letters and starts with " + name.substring(0, 1));
```

Output:

What is your name? Dennis

DENNIS has 5 letters and starts with D

- The nextLine() method reads a line of input as a String.

```
System.out.print("What is your address? ");
String address = console.nextLine();
```


Comparing strings

- Relational operators such as `<` and `==` fail on objects.

```
Scanner console = new Scanner(System.in);
System.out.print("What is your name? ");
String name = console.next(); // input James
if (name == "James") {
    System.out.println("I love you, you love me,");
    System.out.println("We're a happy family!");
}
```

- This code will compile, but it will not print the song.
- `==` compares objects by *references* not the *value*, so it often gives `false` even when two `Strings` have the same letters.

The equals method

- Objects are compared using a method named `equals`.

```
Scanner console = new Scanner(System.in);
System.out.print("What is your name? ");
String name = console.next(); // input James
if (name.equals("James")) // this is TRUE
{
    System.out.println("I love you, you love me,");
    System.out.println("We're a happy family!");
}
```

- Technically this is a method that returns a value of type `boolean`, the type used in logical tests.

String test methods

Method	Description
<code>equals(str)</code>	whether two strings contain the same characters
<code>equalsIgnoreCase(str)</code>	whether two strings contain the same characters, ignoring upper vs. lower case
<code>startsWith(str)</code>	whether one contains other's characters at start
<code>endsWith(str)</code>	whether one contains other's characters at end
<code>contains(str)</code>	whether the given string is found within this one

```
String name = console.nextLine(); // input Dr. Strange
if (name.startsWith("Dr.")) {
    System.out.println("Will you marry me?"); // this will execute
} else if (name.equalsIgnoreCase("butter")) { // input is butters
    System.out.println("You're grounded, young man!"); // this will
    execute
}
```



char

Type char

- **char** : A primitive type representing single characters.
 - A `String` is stored internally as an array of `char`

`String s = "nachos";`

<i>index</i>	<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>
<i>value</i>	'n'	'a'	'c'	'h'	'o'	's'

- It is legal to have variables, parameters, returns of type `char`
 - surrounded with apostrophes: `'a'` or `'4'` or `'\n'` or `'\''`

```
char initial = 'J';  
System.out.println(initial);           // J  
System.out.println(initial + " Joyce"); // J Joyce
```

The charAt method

- The `char` in a `String` can be accessed using the `charAt` method.
 - accepts an `int` index parameter and returns the `char` at that index

```
String food = "cookie";  
char firstLetter = food.charAt(0);    // 'c'  
System.out.println(firstLetter + " is for " + food);
```

- You can use a `for` loop to print or examine each character.

```
String major = "CSE";  
for (int i = 0; i < major.length(); i++) {    // output:  
    char c = major.charAt(i);                // C  
    System.out.println(c);                    // S  
}                                              // E
```


Comparing `char` values

- You can compare `char` values with relational operators:

`'a' < 'b'` and `'X' == 'X'` and `'Q' != 'q'`

- An example that prints the alphabet:

```
for (char c = 'a'; c <= 'z'; c++) {  
    System.out.print(c);  
}
```

- You can test the value of a string's character:

```
String word = console.next();  
if (word.charAt(word.length() - 1) == 's') {  
    System.out.println(word + " is plural.");  
}
```

char VS. String

- "h" is a String, but 'h' is a char (they are different)
- A String is an object; it contains methods.

```
String s = "h";  
s = s.toUpperCase();           // "H"  
int len = s.length();          // 1  
char first = s.charAt(0);      // 'H'
```

- A char is primitive; you can't call methods on it.

```
char c = 'h';  
c = c.toUpperCase();           // ERROR  
s = s.charAt(0).toUpperCase(); // ERROR
```


char VS. int

- Each `char` is mapped to an integer value internally
 - Called an **ASCII value**

'A' is 65

'B' is 66

' ' is 32

'a' is 97

'b' is 98

'*' is 42

- Doing "math" on a `char` causes automatic conversion to `int`.
'a' + 10 is 107, 'A' + 'A' is 130
- To convert an `int` into the equivalent `char`, type-cast it.
(char) ('a' + 2) is 'c'



(Optional) `printf`

Formatting text with `printf`

```
System.out.printf("format string", parameters);
```

- A format string can contain *placeholders* to insert parameters:

- `%d` integer
- `%f` real number
- `%s` string

- these placeholders are used instead of `+` concatenation

- Example:

```
int x = 3;
int y = -17;
System.out.printf("x is %d and y is %d!\n", x, y);
                // x is 3 and y is -17!
```

- `printf` does not drop to the next line unless you write `\n`

printf width

- `%Wd` integer, **W** characters wide, right-aligned
- `%-Wd` integer, **W** characters wide, *left*-aligned
- `%Wf` real number, **W** characters wide, right-aligned
- ...

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 10; j++) {  
        System.out.printf("%4d", (i * j));  
    }  
    System.out.println();    // to end the line  
}
```

Output:

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30

printf precision

- `%.Df` real number, rounded to **D** digits after decimal
- `%W.Df` real number, **W** chars wide, **D** digits after decimal
- `%-W.Df` real number, **W** wide (left-align), **D** after decimal

```
double gpa = 3.253764;
```

```
System.out.printf("your GPA is %.1f\n", gpa);
```

```
System.out.printf("more precisely: %8.3f\n", gpa);
```

Output:

```
your GPA is 3.3
```

```
more precisely:
```

```
          3
        {
3.254
{
8
```

StringBuilder vs String

- StringBuilder is a class that represents a String-like object
- It is made of a sequence of characters, like a String
- The difference between String and StringBuilder objects is that:
 - StringBuilder includes methods that can modify the StringBuilder object once it has been created by appending, removing, replacing, or inserting characters
 - Once created, a String is immutable (cannot be changed) it is replaced by a new String instead

StringBuilder vs String

- It is not possible to make modifications to a String
- Methods used to “modify” a String actually create a new String in memory with the specified changes, they do not modify the old one
- This is why StringBuilders are much faster to work with: they can be modified and do not require you to create a new String with each modification

StringBuilder vs String

```
class StringBuildervsString {  
    public static void main(String[] args) {  
        //create a new StringBuilder object named str  
        StringBuilder str = new StringBuilder("Test immutability");  
        //create a new String object named str2  
        String str2 = "Test immutability";  
  
        //Display the identifying hashCodes for each object  
        System.out.println("StringBuilder: " + str.hashCode());  
        System.out.println("String: " + str2.hashCode());  
  
        //change the values stored in each object  
        str.replace(0, str.length(), "HelloWorld");  
        str2="HelloWorld";  
  
        //Display the identifying hashCodes for each object  
        System.out.println("StringBuilder: " + str.hashCode());  
        System.out.println("String: " + str2.hashCode());  
    }  
}
```